Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller

Software Architecture

Reference	TBD **[TBD-1]**
Issue / revision	Issue 1, Revision 0 — DRAFT
Date	11 October 2026
Author	Andrew Toma
Open items	3 TBD, 6 TBR (App. A)

Document change record

Issue	Rev.	Date	Change
1	0	2026-10-11	First draft

1.0 Purpose and Scope

This document records the software architecture of the System: its components, what each does, how they connect, and the design decisions behind them [1, Sec. 4.2.3, p. 33]. It covers the three software items of the requirements [6, Sec. 3.1].

The architecture concentrates on "the most pressing risks" [3, Sec. 4]: the eight catastrophic and critical hazards that do not yet meet the fault tolerance rule [5, Sec. 5.0]. The software qualities it is built for are the two a high-risk project emphasises, fault tolerance and security [3, Sec. 4], and the testability the requirements demand.

The architecture remains strictly hardware-agnostic, defining functional component behavior rather than the hardware that executes it, and concludes with the capabilities the hardware must provide. Detailed component designs, including specific algorithms and data structures, are documented before each component is implemented [1, Sec. 4.3.2, p. 34].

How to read it. Section 3.0 shows every component and one update on two figures. Section 5.0 holds the design decisions, each with the options weighed and the hazards it targets. Section 6.0 gives each component's role and the requirements it meets. Section 8.0 lists what the hardware must provide.

2.0 Documents

[1] NASA, *NASA Software Engineering Requirements*, NPR 7150.2D.
[2] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, Topic 5.13 "SwDD – Software Design Description".
[3] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, SWE-057 "Software Architecture".
[4] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Concept of Operations*, Issue 1, Revision 1, Draft.
[5] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Hazard Analysis*, Issue 1, Revision 0, Draft.
[6] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Software Requirements Specification*, Issue 1, Revision 0, Draft.
[7] *Fail-Safe In-Orbit Reprogramming of a Spacecraft Controller — Software Classification*, Issue 1, Revision 1, Draft.
[8] NASA, *NASA Software Safety Guidebook*, NASA-GB-8719.13, 2004.

[9] MCUboot project, *Bootloader* design document, `docs/design.md`, commit 1d2643c, 11 September 2026.
[10] G. Reeves and T. Neilson, "The Mars Rover Spirit FLASH Anomaly", IEEE Aerospace Conference, paper 1426, 2005.
[11] NASA, *Space System Protection Standard*, NASA-STD-1006A, 15 July 2022.
[12] NASA, *Fault Management Handbook*, NASA-HDBK-1002, Draft 2, 2 April 2012. A draft, not a released standard.
[13] MCUboot project, *Serial recovery*, `docs/serial_recovery.md`, commit 1d2643c, 11 September 2026.

3.0 System Overview

The System has three software items [6, Sec. 3.1], each made of components (Figure 1). The ground software builds and checks the operator's commands, singly or as a planned sequence. It checks each firmware release the operator approves, encrypts and signs it, and checks the signed result before sending it. It keeps every report that comes back. The onboard computer software checks everything that arrives and passes it on; between contacts it stores the controller's reports, watches the controller, and restores it when no firmware can run. The controller firmware acts on commands, reports its state and handles its own faults. It tries new firmware before keeping it and returns to working firmware on its own when the new firmware fails: a bootloader chooses which copy runs, and the last-resort firmware takes over when the main firmware cannot. Figure 2 follows one update from release to confirmed firmware.

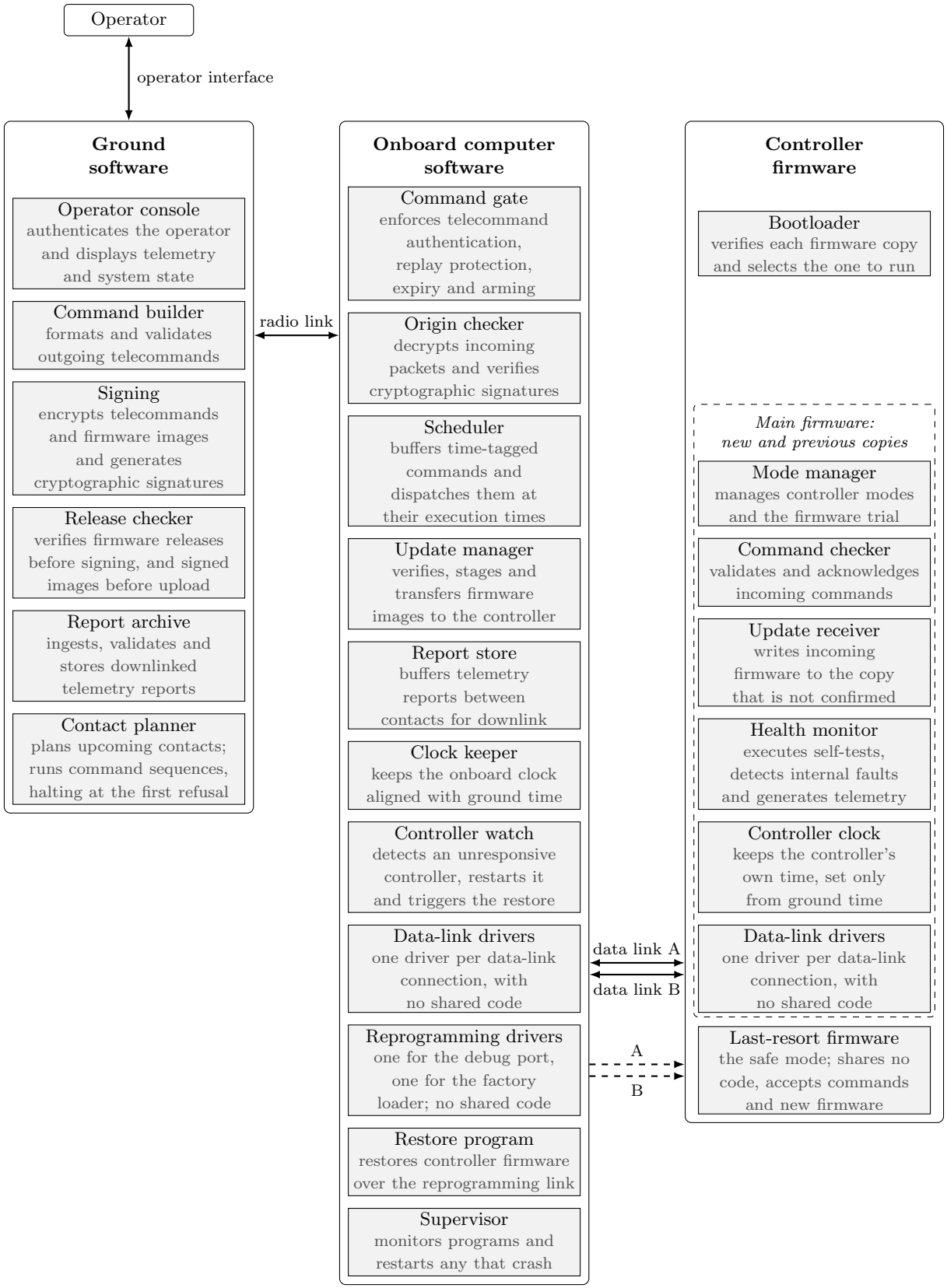


Figure 1: The three software items and their components. The data link and the reprogramming link each have two independent connections, A and B. The dashed arrows are the reprogramming link, which reaches the controller whatever its memory holds.

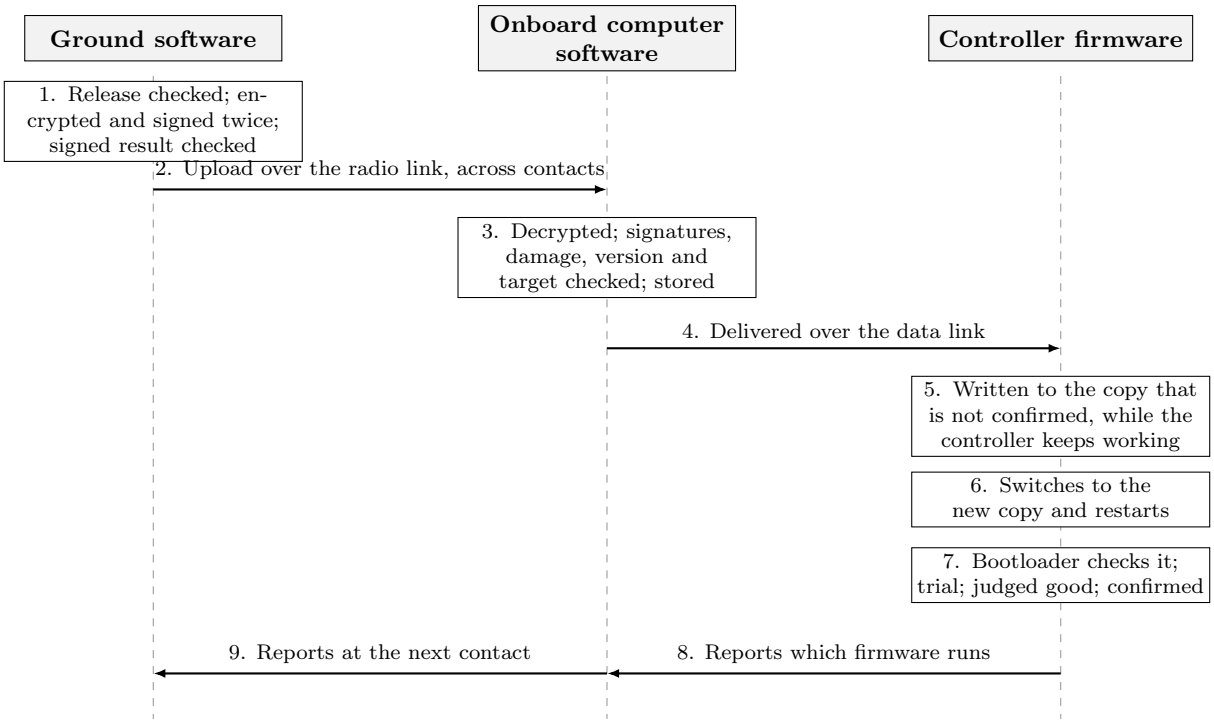


Figure 2: One update, from release to confirmed firmware. If the trial fails, the controller returns to the previous copy (Figure 3).

4.0 Constraints

Three constraints of the ConOps change the design [4, Secs. 1.1.2 and 8.0].

C-3 System development is conducted by a single engineer. Component independence is achieved through isolated software codebases rather than organizational team separation; one engineer writing every codebase limits their diversity.

R-4 The space segment relies on a single onboard computer. A spacecraft can carry a second, redundant one, as the Mars Reconnaissance Orbiter does [12, App. F.2.15, Lesson 2041, p. 197]; how many is a spacecraft decision, outside this project. Its permanent failure halts command handling, telemetry reporting and firmware restoration. This single-point failure risk is formally accepted, so H-4 does not meet the fault tolerance rule [5, Sec. 5.2].

C-4 The onboard computer software and the controller firmware execute on two given development boards. The architectural design remains hardware-agnostic, with board compatibility evaluated separately at the end of this document.

5.0 Design Decisions

Each decision targets hazards that do not yet meet the fault tolerance rule [5, Sec. 5.2]. It states the problem, weighs the options, and lists the requirements the choice adds; these enter the requirements at their next revision. Where a proven design exists, it is one of the options [3, Sec. 4].

Options are compared by the smallest number of failures that, together, cause the hazard with that option in place. A catastrophic hazard needs at least three, a critical one two [5, Sec. 3.3]. The counts leave out the permanent loss of the onboard computer, an accepted risk (R-4). They are design targets, which the hazard analysis confirms in its design phase [5, Sec. 1.0]. In each

options table, the last column marks the option the decision picks (YES) and the options it rejects (NO).

5.1 Start-Up and Fall-Back

Problem. Every firmware copy is started by the same bootloader and depends on the data it keeps over restarts. A defect in the bootloader, or kept data it cannot handle, stops every copy, and today's restore writes back the same bootloader, so the defect returns [5, Sec. 5.2]. One failure is then enough for H-8, the controller stopping between contacts with nothing on board to bring it back, and for H-1, the controller left with no firmware it can run and no way to restore it.

	Option	H-8 CRITICAL	Chosen
a	No bootloader: the last-resort firmware is the first code to run and decides which copy runs. A defect in it stops every start, and the restore puts the same firmware back, so the controller stays down until the ground loads a correction at the next contact.	1	NO
b	MCUboot's design [9, Sec. Boot Swap Types]: a small bootloader runs first, runs a new copy as a test and returns to the old copy unless the new one confirms itself. It survives a defect in any copy, but not one in the bootloader itself: that stops every start, and the restore writes the same bootloader back.	1	NO
c	Option b, plus a last-resort firmware sharing no code with the bootloader or the copies. When the bootloader fails, the controller goes silent and the onboard computer restores it, putting the last-resort firmware where the bootloader was, so it runs first (Figure 4). The controller stays down only if a second failure follows: a defect in the last-resort firmware, or a failed restore.	2	YES

Choice. Option c, shown in Figures 3 and 4. It keeps MCUboot's test and revert, which exists to keep bad firmware from leaving a device unusable [9, Sec. Boot Swap Types], and adds a last-resort firmware sharing no code with the rest [6, REQ-009]. The last-resort firmware is the controller's safe mode: it still accepts commands from the ground [12, Sec. 6.4.3.2.4, p. 85]. H-1, which also needs the ground's way in, is decided in Section 5.2.

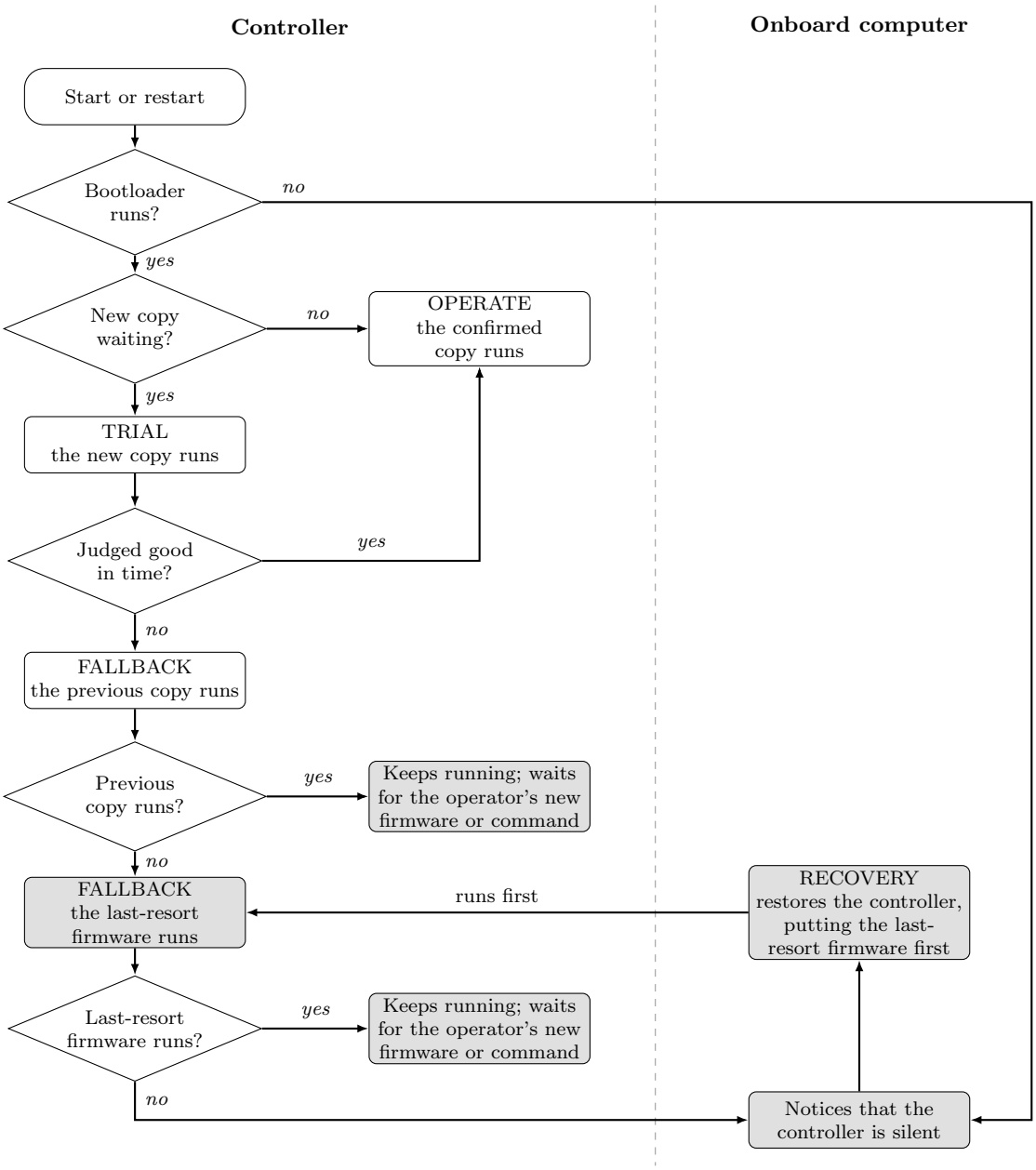


Figure 3: Start-up and fall-back as a flow. Each diamond is a check with a yes and a no exit; a hang or too many failed starts counts as no. White: the two-copy test and revert of MCUboot [9, Sec. Boot Swap Types]. Grey: added for a controller that nobody can reach: the last-resort firmware, the wait for the operator after a fall-back [6, REQ-117], and the restore by the onboard computer.

Note. *The restore is how the onboard computer brings back a controller that runs no firmware. Through the reprogramming link, and without using any controller software, it restarts the controller, writes its own copy of the last-resort firmware into the place the controller starts from, clears the kept data and starts the controller again. It tells the ground at the next contact [6, REQ-054 to REQ-060].*

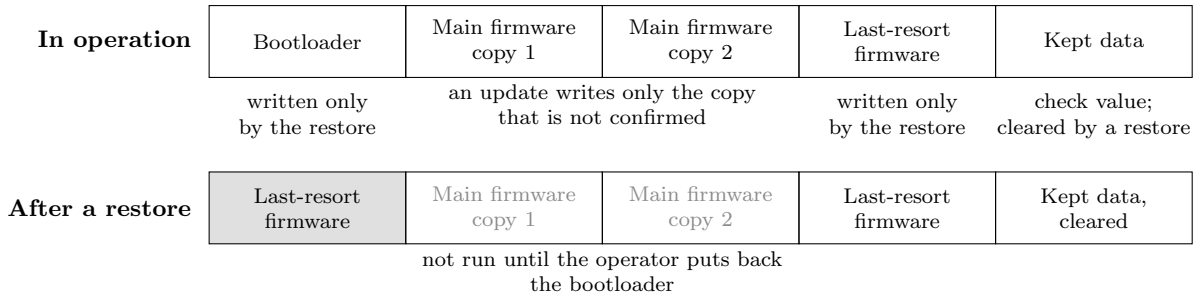


Figure 4: The controller's memory, not to scale. After a restore, the last-resort firmware (grey) sits in the bootloader's place and runs first.

Adds.

- The bootloader shall share no code with the main firmware or the last-resort firmware.
- The restore shall put the last-resort firmware in the bootloader's place, so that it runs first.
- The onboard computer software shall put back the bootloader and the main firmware through the reprogramming link when the operator orders it.

5.2 The Way Back In

Problem. When the bootloader is broken (Section 5.1), the controller runs nothing, and only a restore can bring it back. A restore that needed any software on the controller to receive it would fail at exactly that moment, which the requirements forbid [6, REQ-056]. The restore therefore needs a way into the controller's memory that works with nothing running on it, and the ground needs the same way in to load corrected firmware [6, REQ-058]. Targets H-1, the controller left with no firmware it can run and no way to restore it. The same way in also serves H-3 and H-4.

	Option	H-1 CATASTROPHIC	Chosen
a	The controller is asked to restore itself, as in MCUboot's serial recovery, which runs inside the bootloader [13, Sec. Entering Serial Recovery Mode]. A broken bootloader means nothing receives the order and nothing can load a correction.	1	NO
b	One separate way in that needs nothing running on the controller; the onboard computer drives it and can restart the controller. A broken bootloader is bypassed, but if this way in also breaks, nothing reaches the controller.	2	NO
c	Two separate ways in that share nothing: the controller's debug port, which writes memory directly with the processor stopped, and its factory loader, which a boot pin starts instead of our bootloader. A broken bootloader and one broken way in still leave the other. Each way in has its own driver on the onboard computer; a defect in the restore program that uses both is corrected by the system outside this project (A-6), as for H-3.	3	YES

Choice. Option c **[TBR-1]**. It follows the principle of hardware commands, which a spacecraft executes on receipt without any flight software [11, Sec. 3.2, p. 9]. Such a command let the Spirit rover's team regain control while its flight software kept resetting: it made the software start without its flash memory [10, pp. 4 and 9]. Two ways in are the fewest that meet the rule. They still share the controller's memory and its reset wire; the hazard analysis counts these shared causes when it is re-run on this design.

Adds.

- The reprogramming link shall have two connections, independent of each other: the controller's debug port and its factory loader, each able to load the controller's memory whatever that memory holds.
- The onboard computer software shall drive the debug port and the factory loader with separate drivers that share no code.
- The onboard computer software shall be able to restart the controller.
- No controller firmware shall be able to disable the debug port or the factory loader.

5.3 Restore Apart From Update

Problem. New firmware can reach the controller in two ways. In the normal update, the onboard computer sends it over the data link to the controller's running firmware, which writes it into memory. In the restore, the onboard computer writes it into the controller's memory itself, through the reprogramming link [6, REQ-058]. Both start on the onboard computer, from a copy of the firmware stored there. If one program did both jobs from one stored copy, a single defect in that program, or a single damaged copy, would block both ways at once. New firmware could then reach the controller only after the system outside this project corrects the onboard computer's software (A-6) [5, Sec. 5.2]. Targets H-3, new firmware no longer able to reach the controller.

	Option	H-3 CATASTROPHIC	Chosen
a	One program does both the update and the restore, and both read the same stored copy of the firmware. One defect in that program, or damage to that copy, blocks both ways; the only barrier left is the outside system correcting the onboard computer (A-6).	2	NO
b	Two separate programs, the update manager and the restore program, each with its own code and its own stored copy of the firmware. Blocking both ways takes a defect in each program, and the outside correction failing as well.	3	YES

Choice. Option b [**TBR-2**]. The two programs share nothing that one defect could reach: when the update fails, the restore still works, and the other way round.

Adds.

- The restore program shall share no code or stored data with the update manager.

5.4 Proof of Origin

Problem. One means proves that both commands and firmware come from the operator. If it is disclosed, another party can command the controller and load firmware [5, Sec. 5.2]. Targets H-2, firmware from someone else running on the controller, and H-5, someone else commanding the System.

	Option	H-2 CATASTROPHIC	H-5 CATASTROPHIC	Chosen
a	One secret proves both commands and firmware, stored on the ground and on board. Disclosing it from either place hands over both.	1	1	NO
b	A digital signature made with one key kept on the ground; the spacecraft holds only the means to check it, as in MCUboot [9, Sec. Security]. Nothing on board can be disclosed, but one disclosed key on the ground still hands over both.	1	1	NO
c	Every command and firmware image is also encrypted, as the command stack must be [11, Sec. 4.1.1, p. 9]. The encryption key and the signing key are kept in two separate places, so another party needs both.	2	2	NO
d	As c, with a second signature made with a third key kept in a third place. Another party needs all three keys.	3	3	YES

Choice. Option d [**TBR-3**], shown in Figure 5. The command stack must be encrypted [11, Sec. 4.1.1, p. 9], and encryption with authentication is the recommended defence in depth [11, Sec. 4.1.1.c, p. 10]. The controller checks firmware signatures again before running it [6, REQ-006], with code different from the onboard computer's, so that one defect in that code cannot pass forged firmware through both checks [8, Sec. 7.4.2, p. 141].

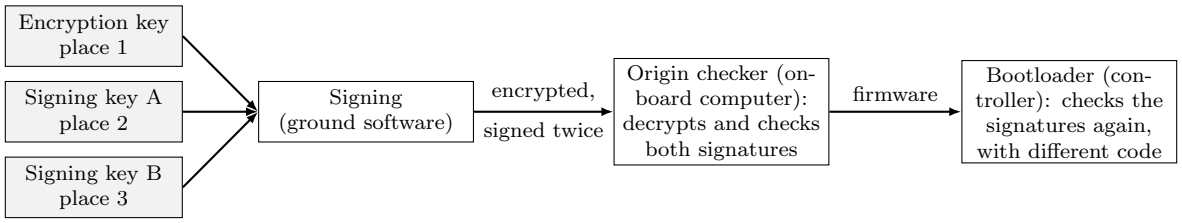


Figure 5: Proof of origin. Each key is kept in its own place, so one disclosed key is not enough. Commands stop at the onboard computer's check; firmware is checked again on the controller.

Adds.

- Every command and firmware image shall be encrypted from the ground software to the onboard computer software [11, Sec. 4.1.1, p. 9].
- Every command and firmware image shall carry two signatures, made with two keys kept apart from each other and from the encryption key.
- The onboard computer software shall act on a command or firmware image only when it decrypts and both signatures are valid.
- The controller firmware shall check firmware signatures with code different from the onboard computer software's.

5.5 Two Clocks

Problem. Expiry, arm time-outs, scheduled commands, time-tags and the trial time limit all depend on the onboard clock. A wrong clock alone runs a scheduled command at the wrong time and gives reports wrong times [5, Sec. 5.2]. Targets H-6, here a scheduled command run at the wrong time, and H-9, here reports shown with wrong times.

	Option	H-6 CRITICAL	H-9 CRITICAL	Chosen
a	One clock, compared with ground time only at contacts [6, REQ-072]. Between contacts, a wrong clock runs scheduled commands at the wrong time and stamps wrong times on reports, and nothing notices.	1	1	NO
b	Report and command numbers replace times where possible. The order of reports survives a wrong clock, but scheduled commands and time-tags still need it.	1	1	NO
c	A second clock in the controller, with its own time source and set only from ground time. A scheduled command runs only when both clocks agree, and every report carries both times, so one wrong clock is caught by the other. A wrong ground time reaches both clocks; the ground software's check of every command before it is sent [6, REQ-089] keeps that case at two failures, and a wrong time that passes it is an accepted risk.	2	2	YES

Choice. Option c [**TBR-4**], shown in Figure 6.

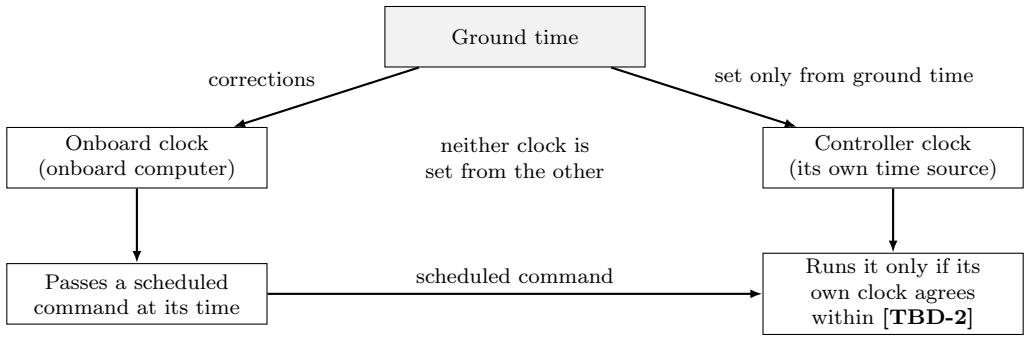


Figure 6: Two clocks. Neither is set from the other, so one wrong value cannot reach both.

Adds.

- The controller firmware shall keep its own clock, from a time source separate from the onboard computer's.
- The controller's clock shall be set only from ground time.
- The controller firmware shall refuse a command past its expiry, by its own clock.
- The controller firmware shall run a scheduled command only when its own clock agrees with the scheduled time within [**TBD-2**].
- Every report shall carry the time of both clocks.

5.6 Held Data

Problem. A bit error can change a held command, the arm state or the kept data after its check and before its use (A-5) [5, Sec. 5.2]. Targets H-6, here a command acted on in a form the operator did not intend.

	Option	H-6 CRITICAL	Chosen
a	Each item is checked once, on arrival. A bit error afterwards goes unnoticed.	1	NO
b	Each held item carries a check value and is checked again just before use; the arm state is stored as a pattern that no single bit error turns into armed. A bit error now has to produce a matching check value as well.	2	YES
c	Memory hardware that corrects bit errors on its own. Also two failures, but it depends on a hardware feature.	2	NO

Choice. Option b **[TBR-5]**. MCUboot notes the same risk: skipping the check before use lets an image changed after a good check run unchecked [9, Sec. Integrity Check]. Unlike option c, option b needs no hardware feature; option c can be added where the hardware has it.

Adds.

- Every held command and the kept data shall be stored with a check value and checked just before use.
- The arm state shall be stored so that no single bit error makes it armed.

5.7 Two Kinds of Connection

Problem. The two data-link connections share software at both ends, so one defect cuts both (R-1) [5, Sec. 5.2]. Targets H-4, the controller no longer commanded or observed. The count is of failures that cut both connections at once.

	Option	Failures to cut both	Chosen
a	Two connections of one kind share one driver at each end. One defect in that driver cuts both.	1	NO
b	Each connection has its own driver, but both are of one kind. A fault common to that kind, in its design or in its handling, cuts both.	1	NO
c	Two different kinds, each with its own driver. Cutting both takes a separate failure in each.	2	YES

Choice. Option c **[TBR-6]**. One connection carries on when the other is cut (O-3). H-4 still falls to the loss of the onboard computer (R-4).

Adds.

- The two data-link connections shall be of different kinds.

6.0 Components

Each component, its role, and the requirements it meets [2, Sec. 3.3]. Every requirement [6, Secs. 4.0–7.0] is met by at least one component; requirements that the decisions add are allocated when they enter the requirements.

All components are safety-critical [7, Sec. 4.0] and new code, except the bootloader, the origin checker and signing, which are built around a published cryptographic library.

The components from the mode manager to the data-link drivers form the main firmware; the controller keeps two copies of it. Figure 3 shows which copy runs in each controller mode [4, Sec. 3.4].

Component	Role	Requirements
Controller firmware		
Bootloader	Runs first at every start. Sets a safe state, checks each firmware copy for damage and origin, counts failed starts, and starts the new, previous or last-resort firmware.	REQ-004 to 006, REQ-013 to 019, REQ-023, REQ-113
Mode manager	Holds the controller mode, runs the trial, and confirms new firmware or gives it up.	REQ-001 to 003, REQ-011 to 013, REQ-020, REQ-021, REQ-024 to 026, REQ-113, REQ-117
Command checker	Checks every command for mode, values, arming, expiry and repeats; checks held commands again before running them; answers and logs.	REQ-027 to 036, REQ-100
Update receiver	Stores new firmware in the free copy, never over the previous or last-resort firmware.	REQ-007, REQ-008
Health monitor	Tests the controller, detects faults, restarts it when it stops responding, and sends health and event reports.	REQ-022, REQ-037 to 041
Controller clock	Keeps the controller's own time, set only from ground time.	REQ-030
Data-link drivers	One driver for each data-link connection, sharing no code.	REQ-042, REQ-099
Last-resort firmware	A separate program sharing no code or kept data with the others. Sets a safe state, answers commands, sends reports, and receives new firmware.	REQ-004, REQ-009, REQ-010
Onboard computer software		
Command gate	Checks every command for origin, newness, expiry and arming, passes it on, and logs it.	REQ-064 to 067, REQ-087, REQ-100, REQ-112
Origin checker	Decrypts commands and firmware, checks their two signatures, and replaces the means of checking them on command.	REQ-043, REQ-064, REQ-084 to 086
Scheduler	Holds scheduled commands and passes each on once, at its run time, across restarts.	REQ-068, REQ-069
Update manager	Checks new firmware, stores it, and delivers it over the data link, resuming after any interruption.	REQ-043 to 053, REQ-063
Report store	Checks, numbers and time-tags every report, stores reports between contacts, and sends them at the next one.	REQ-070, REQ-071, REQ-074 to 079
Clock keeper	Compares the onboard clock with ground time and applies corrections.	REQ-072, REQ-073
Controller watch	Notices a silent controller, restarts it, and calls the restore when no firmware runs.	REQ-061
Supervisor	Restarts any program of the onboard computer software that crashes.	REQ-062
Data-link drivers	One driver for each data-link connection, sharing no code.	REQ-080 to 082, REQ-099
Reprogramming drivers	One driver for the controller's debug port and one for its factory loader, sharing no code.	REQ-054, REQ-058, REQ-059, REQ-083
Restore program	A separate program with its own firmware store. Restores the controller through the reprogramming drivers, loads firmware from the ground through them, and checks its store and both ways in between uses.	REQ-054 to 060, REQ-083
Ground software		
Operator console	Lets only the operator in, and shows reports, unanswered commands and the running firmware.	REQ-091, REQ-094, REQ-095
Command builder	Builds every command from the shared command list and checks it before it is sent.	REQ-089, REQ-100
Signing	Encrypts and signs every command and firmware image with three keys kept in separate places, and keeps a recoverable copy of the means of proving origin.	REQ-088, REQ-096
Release checker	Checks every firmware release before signing, and the signed image before it is sent, with the checks the onboard computer makes.	REQ-090, REQ-114
Report archive	Keeps every report received and checks it for damage.	REQ-092, REQ-093
Contact planner	Plans commands, uploads and report retrieval for coming contacts, and sends a planned command sequence, stopping at the first command refused or not answered.	REQ-097, REQ-115, REQ-116

Component	Role	Requirements
All software items		
Every component	Rules that apply to all software.	REQ-098, REQ-101 to 111

7.0 Interfaces

Each interface is specified byte by byte before any code is written. Interfaces between components of one software item are set in its detailed design.

Interface	Between	Carries
Operator interface	Operator and ground software	The operator's decisions; reports and the System's state
Radio link	Ground software and onboard computer software	Signed commands and firmware; answers and reports
Data link A	Onboard computer and controller	Commands, answers, reports and firmware
Data link B	Onboard computer and controller	The same as data link A, over a different kind of connection
Reprogramming link A	Reprogramming drivers and controller	The controller's debug port: its restart, and memory writes with the processor stopped
Reprogramming link B	Reprogramming drivers and controller	The controller's factory loader: started by a boot pin, it accepts new firmware

8.0 Hardware Needs

The hardware must provide what the decisions above assume. Each need is checked against the two boards before this document is released [**TBD-3**].

Need	Needed by
A debug port that writes the controller's memory whatever it holds	Section 5.2
A factory loader in the controller that firmware cannot change, started by a boot pin	Section 5.2
Wires from the onboard computer to the controller's reset and boot pins	Section 5.2
No way for controller firmware to disable the debug port or the factory loader	Section 5.2
A way for the restore to make the last-resort firmware run first	Section 5.1
Room in the controller for the bootloader, two main firmware copies, the last-resort firmware and the kept data	Section 5.1
Kept data in the controller that survives a restart and a power loss	REQ-017
Room on the onboard computer for two separate firmware stores	Section 5.3
Processing in the controller to check two signatures within the restart time	Section 5.4; REQ-023
A time source in the controller separate from the onboard computer's	Section 5.5
Two different kinds of connection between the onboard computer and the controller	Section 5.7

A Open Items

The values below are not yet known.

TBD-1 Project name and document reference.

TBD-2 Largest allowed difference between the controller's clock and the time of a scheduled command.

TBD-3 Check of each hardware need against the two boards.

The proposals below await the author's acceptance.

TBR-1 The way back in: option c (Section 5.2).
TBR-2 Restore apart from update: option b (Section 5.3).
TBR-3 Proof of origin: option d (Section 5.4).
TBR-4 Two clocks: option c (Section 5.5).
TBR-5 Held data: option b (Section 5.6).
TBR-6 Two kinds of connection: option c (Section 5.7).

The author accepted, on 11 October 2026, option c of Section 5.1 and the risk of a wrong ground time that passes the ground software's check (Section 5.5).